AI Engineering and Claude Architect · Part I, Foundations

Working With Claude, Lesson 6

Context Engineering

At turn 3, the customer said "$117.42, by Friday". By turn 40, your agent's summary says "a small refund, soon". The customer is furious, and technically no component failed. What you choose to keep in front of the model matters as much as anything you ask it.

Module	AI03 of 13
Lesson	6 of 6, including the module revision
Prerequisites	AI03 L1–L5, AI02 L6
Exam relevance	**Direct.** TS 5.1 in full, and a foundation for TS 5.4 and TS 1.7. Scenarios 1, 2 and 3 use it often
Reading time	~45 minutes

Contents

1. Context Engineering
2. Module Revision

Lesson 6 of 6 · Working With Claude

Context Engineering

Key idea

Context is a finite resource, so treat each token as a design decision. Prompt engineering asks how to write instructions. Context engineering asks what should be in front of the model right now. **Keep exact facts outside summaries, trim tool results before they accumulate, put what matters where it will be noticed, and move detail out of the window**, into notes, subagents and retrieval.

> *Before: position effects (AI02 L6), stateless history (AI03 L1), caching (AI03 L5). Teaches: attention budgets, case facts, trimming, ordering, summarization risks, just-in-time retrieval, compaction, context editing, notes, and subagents. Used by: AI05 L4, AI06 L6–L7, AI12 and AI13 L6.*

The hook: the summary that lost the money

Your support agent handles a long conversation. To save tokens, it summarizes after every 10 turns.

Turn	What the customer said	What the summary kept
3	"Order ORD-1 was $117.42. I need the refund by Friday, since my rent's due."	"Customer wants a refund for a recent order."
12	"I was also double-charged $23.10 on ORD-2."	"Customer has a couple of billing issues."
40	"So will I get it by Friday?"	Agent: "Refunds usually take 5–7 days."

Each summary was reasonable English. Together they erased every number that mattered.

Key idea

Progressive summarization destroys exactly what transactions depend on. TS 5.1 names the risk directly: condensing numerical values, percentages, dates and customer-stated expectations into vague summaries.

Intuition: a desk, a filing cabinet and sticky notes

Picture a claims handler at a desk.

Place	What's kept there	Equivalent
Sticky notes on the monitor	Exact facts for this case: the amount, the date, the order number	Case facts block
The desk	What they're actively working on	Context window
The filing cabinet	Everything else, fetched when needed	Tools, retrieval, notes
An assistant	Reads the bulky file, and reports back a summary	Subagent

A good handler keeps the desk clear. A desk piled with every printout doesn't make them smarter, and it makes them miss the sticky note.

Prompt engineering and context engineering

Anthropic's post Effective Context Engineering for AI Agents draws the distinction:

	Prompt engineering	Context engineering
Concern	Writing and organizing instructions	Curating all the tokens at inference
When	Mostly a discrete design task	**Iterative**, at each model call
Includes	System prompt	System prompt, tools, history, retrieved data, examples

Its central idea is an **attention budget**: "Every new token introduced depletes this budget by some amount." It describes **context rot**, where recall declines as the window fills. So the goal is the "smallest possible set of high-signal tokens" that makes the desired outcome likely.

Five things compete for the window, and each takes budget from the others:

Occupant	Typical growth
System prompt and tools	Fixed per request
Conversation history	Grows each turn
Tool results	**Usually grows fastest**
Retrieved documents	Varies per request
Room for the reply	Must be reserved

A larger window doesn't remove the problem

Current Claude models have a 1M-token window. That's a capacity limit, and the attention problem is separate. Recall AI02 L6's "lost in the middle" result, and the guide's sample question stating that larger context windows don't solve attention quality. So **room isn't the same as focus.**

Technique 1: A case facts block

Extract transactional facts into a structured block, included in each prompt, and kept **outside** summarized history (TS 5.1).

From `code/ai03-claude/.../ContextAssembler.java`:

```
public record CaseFacts(String customerId, Map<String, String> orders, String stated) {
    String render() {
        StringBuilder sb = new StringBuilder("<case_facts>\n")
                .append("customer: ").append(customerId).append('\n');
        orders.forEach((id, detail) ->
                sb.append("order ").append(id).append(": ").append(detail).append('\n'));
        return sb.append("customer expects: ").append(stated)
                .append("\n</case_facts>").toString();
    }
}
```

Real output of the assembled prompt:

```
<case_facts>
customer: C-8841
order ORD-1: $117.42, delivered 2026-10-30, damaged
customer expects: full refund by Friday
</case_facts>

<earlier_conversation_summary>
Customer reported a damaged order and asked about a refund.
</earlier_conversation_summary>

<recent_turns>
user: Can you confirm the amount?
</recent_turns>

Answer the user's last message.
```

Now the summary can be as short as you like, because the numbers survive independently.

What goes in case facts	What can be summarized
Amounts, dates, IDs, statuses	Pleasantries
Customer-stated expectations	Explanations already given
Decisions made and commitments given	Reasoning that led to them
Unresolved items	Resolved small talk

Who updates the block? Prefer your code, from tool results, for verified values. For customer-stated values, the model can extract them through a tool. Don't let a summary overwrite verified facts.

For multi-issue sessions, keep a separate structured record per issue, as TS 5.1 states.

Technique 2: Trim tool results before they accumulate

Tool results grow fastest, and they often carry far more than the agent needs. Real output, with a 40-field order and rough estimates:

```
fields: 40 -> 5
estimated tokens per lookup: 419 -> 21
 1 lookups:    419 ->     21 estimated tokens
10 lookups:  4,190 ->    210 estimated tokens
50 lookups: 20,950 ->  1,050 estimated tokens
```

```java
static Map<String, String> trim(Map<String, String> toolResult, Set<String> relevant) {
    Map<String, String> kept = new LinkedHashMap<>();
    toolResult.forEach((key, value) -> {
        if (relevant.contains(key)) {
            kept.put(key, value);
        }
    });
    return kept;
}
```

The token numbers are rough, from a character estimate on invented data. The savings ratio is what matters. Where should trimming happen?

Place	How	When
In the tool or MCP server	Return a compact response by default	Best, if you own the tool
In a `PostToolUse` hook	Filter before the model sees it	Tools you don't own (AI06 L8)
With a parameter	`lookup_order(id, fields=[...])`	When the needs vary

Trim by relevance to the agent's role. A returns agent needs the delivery date, not warehouse routing. Don't trim what provenance needs, such as source URLs (AI05 L4).

Technique 3: Put things where they'll be noticed

From AI02 L6, and TS 5.1:

Action	Why
Put a key findings summary at the start of aggregated input	Addresses position effects
Use explicit section headers for detailed results	Helps locate and separate content
Put long data above instructions, with the query last	Anthropic's long-context advice (AI03 L2)
Keep case facts in a consistent, prominent place	Always available

Technique 4: Pass structured data between agents

When one agent's output is another's input, verbose reasoning wastes the downstream budget. TS 5.1 says to modify upstream agents to return **structured data (key facts, citations, relevance scores)** instead of verbose content and reasoning chains, and to include metadata (dates, source locations, methodological context) to support accurate synthesis.

Upstream returns	Downstream cost	Quality for synthesis
Full reasoning transcript	High	Poor signal, and sources may be lost
Short prose summary	Low	Loses numbers and attribution
Structured findings with metadata	Low	Preserves facts and sources

AI06 L6 builds this.

Technique 5: Move work out of the window

Anthropic's post describes retrieval strategies, and three long-horizon techniques.

Just-in-time retrieval

Keep lightweight **references**, such as file paths or queries, and load data only when it's needed. Claude Code works this way: it doesn't read a whole repo upfront. It searches, then opens what's relevant. Often you'll combine this with some up-front retrieval. This approach matches TS 2.5's "building understanding incrementally".

Choosing a long-horizon technique

Technique	How it works	Anthropic's guidance on use	Risk
Compaction	Summarize a nearly full context, and continue	Extended back-and-forth conversation	Losing details, as in the hook
Clearing old tool results	Remove results already used	Described as a low-risk form of compaction	Model must fetch again if needed
Structured notes	Agent writes notes outside the window, reloads them later	Iterative work with clear milestones	Stale or wrong notes persist
Subagents	Focused work in a clean window, returning a condensed result	Parallel research and analysis	Coordination, and lost context

The shape of the work points to the approach, and long-running systems usually combine them. AI06 L6 and L7 go deep on subagents and long-running harnesses, and TS 5.4 tests scratchpad files, subagent delegation and `/compact`.

API-level features

Feature	What it does	Important detail
Compaction (beta)	The API summarizes earlier context near a trigger, by default at 150K tokens	Append `response.content`, including compaction blocks, or the state is lost
Context editing (beta)	Clears old tool results or thinking blocks	Different from compaction, which summarizes
Memory tool	Lets Claude read and write files that you host	Persists across conversations (AI06 L7)
`/compact` in Claude Code	Reduces context during long sessions	TS 5.4 (AI12)

> *Compaction doesn't know what's sacred*
>
> Whether a person, your code or the API creates a summary, it may lose exact values. **That's why case facts live outside it.** Using compaction doesn't remove the need for Technique 1.

How Anthropic solved it

Problem	Anthropic's published approach
Models lose focus as context grows	The attention budget idea, and context engineering as a practice
System prompts that are too rigid or too vague	The "right altitude" (AI03 L2)
Tool bloat	Tools that are "self-contained, robust to error, and extremely clear" (AI06 L3)
Rule lists	A small set of diverse, canonical examples
Long tasks outlive one window	Compaction, notes, and sub-agent architectures, offered as API and Agent SDK features

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: send everything. Full history and full tool results go on each request. Cost and latency grow, and attention degrades long before you hit a limit.

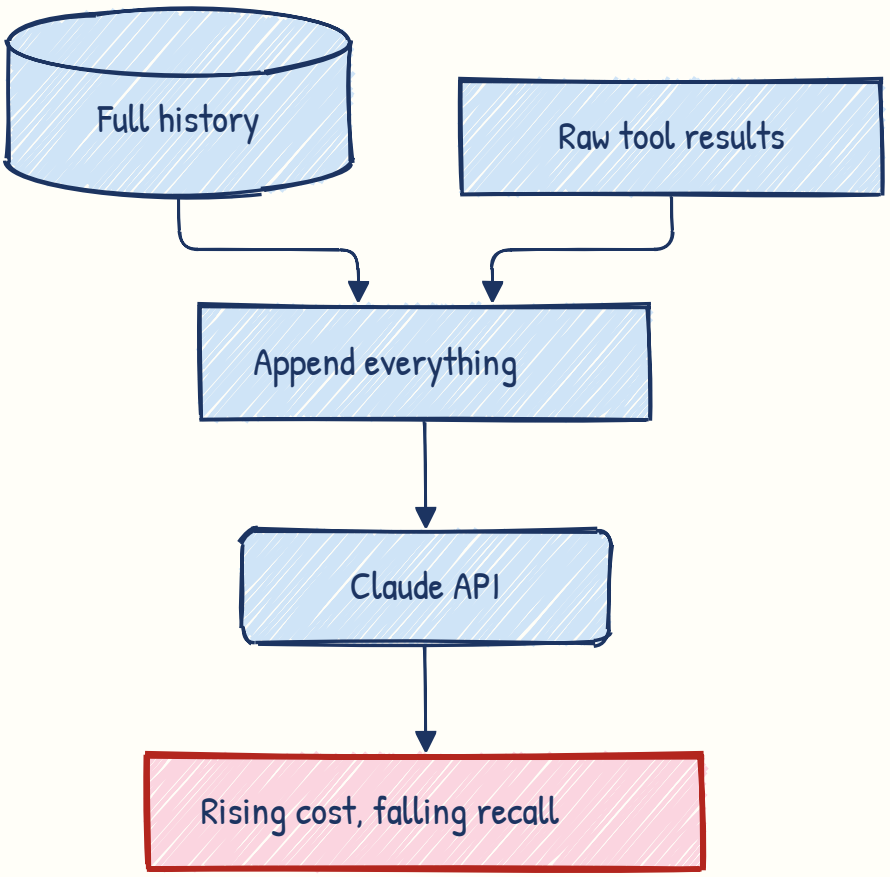


Figure 1. Baseline. History and raw tool output pile up with no curation, until quality or the limit breaks.

Improved: summarize when it's long. Tokens drop. New problem: **the hook's failure: amounts, dates and commitments get blurred, and nothing detects it.**

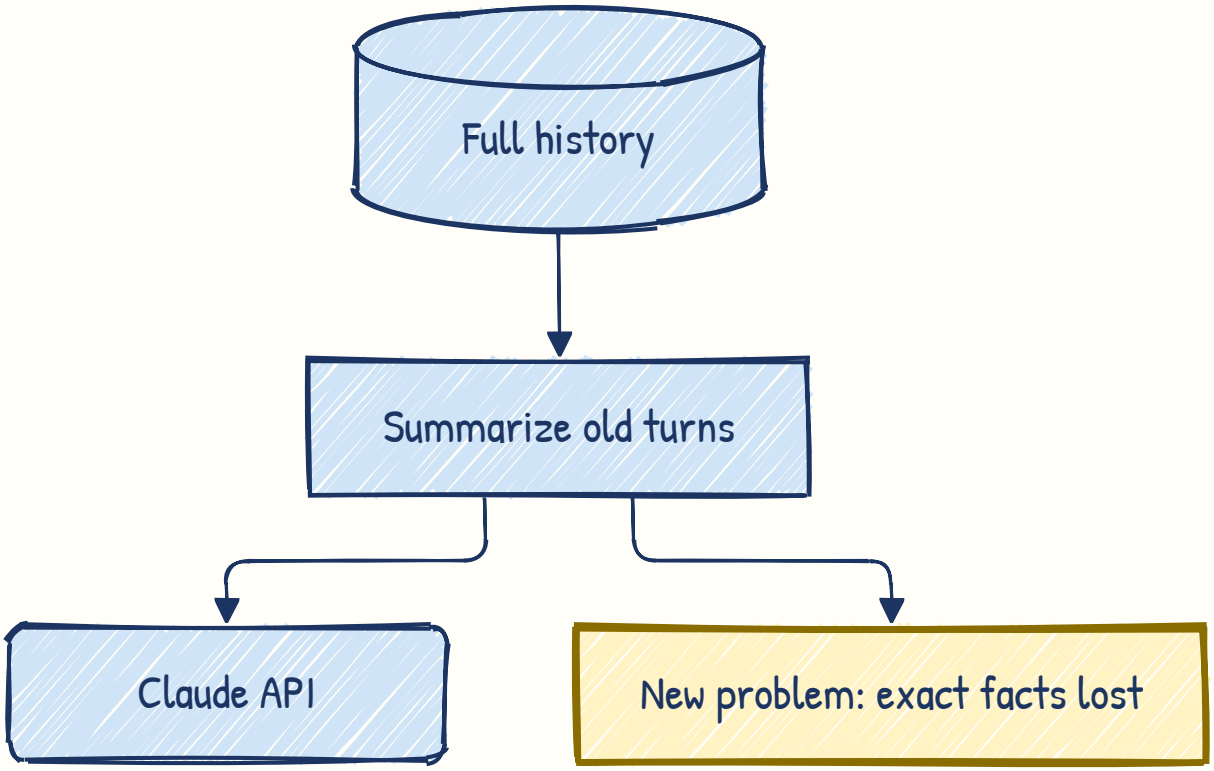


Figure 2. Improved. Summaries control size, but exact facts now depend on what a summarizer chose to keep.

Production: a context assembler, with separate layers.

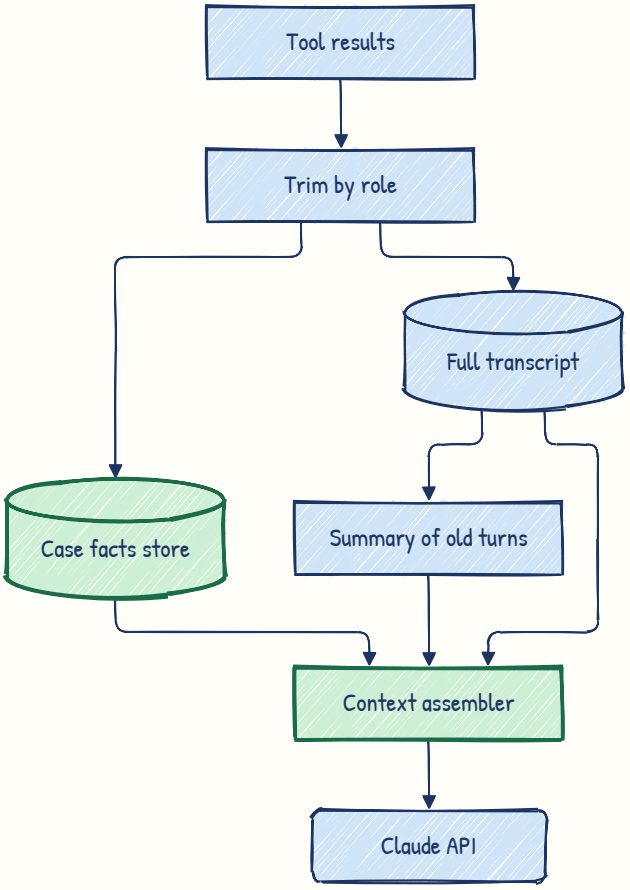


Figure 3. Production. Exact facts, summaries and recent turns are separate layers. Tool results are trimmed on the way in, and an assembler builds each request in a stable order.

Component	Why it exists	How it communicates
Trim step	Stops tool bloat at the source	Tool wrapper or hook
Case facts store	Exact values outside any summary	Database row per session or issue
Full transcript	Audit, and rebuilding summaries	Durable store, never sent in full
Summary	Keeps the gist of old turns	Generated asynchronously
Context assembler	One tested place for order and budget	In-process, before the API call

Cache note (AI03 L5): order layers so that stable ones come first. If facts change on each turn, place them after your last breakpoint, or use a mid-conversation system message on supported models. Editing old messages can also conflict with preserved thinking, so plan this at design time.

Failures

Failure	Effect	Containment
Summary contradicts facts	Agent confusion	Tell the model that case facts take priority
A fact extraction error	A wrong value persists	Prefer verified tool values, and show them to the user to confirm
Trimmed a field the agent needed	Repeat lookups, or wrong answers	Review by role, and offer a `fields` parameter
Old notes are stale	Bad decisions	Date entries, and re-verify before acting
Over-trimming sources	Lost provenance	Treat source metadata as required

Trade-offs

Decision	Gain	Cost
Case facts	Exact recall	Extraction logic to maintain
Aggressive trimming	Lower cost, better focus	May need refetching
Summaries	Long conversations stay affordable	Detail loss
Subagents	Clean windows, isolation	Coordination cost (AI06 L6)
Just-in-time retrieval	Small context	More tool calls and latency
Sending everything	Nothing is lost	Cost and attention

When you don't need any of this: short conversations with small tool results. Start simple, and add a layer when you measure a problem.

Common mistakes

1. **Summarizing amounts and dates.** Keep them in case facts.
2. **Sending raw tool output.** 40 fields when 5 are needed wastes budget.
3. **Treating a larger window as the fix.** Capacity isn't attention.
4. **Burying key facts in the middle.** Put a summary first, and use headers.
5. **Passing reasoning transcripts to the next agent.** Pass structured findings.
6. **Dropping source metadata during trimming.** You'll lose attribution.
7. **Letting a summary overwrite verified data.** Facts have priority.
8. **Rewriting history casually.** Check it against preserved thinking and caching.

What to say in an interview

"I treat context as a budget. Exact facts go in a structured block outside any summary, tool results get trimmed by role before they accumulate, and key findings go first. Long work moves out of the window using notes or subagents. A bigger window helps capacity, not attention, so I'd measure recall rather than assuming it."

Certification lens

How this lesson maps to CCAR-F

Direct: all of **TS 5.1**. Foundations for **TS 5.4** (scratchpads, subagents, `/compact`) and **TS 1.7**.
Likely wrong answers: a bigger context window, summarizing more often, or removing history altogether.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Customer Support Resolution Agent · Medium. In long sessions, the agent misstates refund amounts and dates that customers gave earlier. The system summarizes old turns to save tokens. What's the best fix?

A. Summarize more often, so that summaries stay fresh.

B. Extract amounts, dates and order numbers into a persistent facts block, kept outside the summarized history.

C. Ask customers to restate their details on each turn.

D. Upgrade to a model with a larger context window, and stop summarizing.

A **B** (TS 5.1). A loses detail faster. C harms the experience. D may delay the problem, and it costs more without addressing attention.

Q Q2 · Exam style, scenario: Multi-Agent Research System · Hard. A synthesis agent runs out of context budget when receiving subagent reports, and attribution is getting lost. Choose the **two** most effective changes.

A. Have upstream agents return structured findings: key facts, citations and relevance scores.

B. Require metadata such as dates and source locations in those outputs.

C. Pass each subagent's complete reasoning transcript, so that nothing is lost.

D. Summarize all findings into one paragraph before synthesis.

E. Remove the citations, to save tokens.

A **A and B** (TS 5.1, TS 5.6). C makes the budget problem worse. D and E destroy the attribution that the system needs.

Q Q3 · Exam style, scenario: Multi-Agent Research System · Medium. Reports consistently omit findings placed in the middle of a long aggregated input. What change most directly addresses this?

A. Repeat the instruction to include all findings.

B. Place a key findings summary at the start, and organize details under explicit section headers.

C. Reverse the order of findings on each run.

D. Increase `max_tokens`.

A **B** (TS 5.1). This addresses position effects directly. A is vague. C only moves the problem. D affects output length, not what's noticed in the input.

Q Q4 · Match the following · Medium. Match each situation to a technique. Situations: (1) a long customer chat, (2) a coding task across many sessions with milestones, (3) parallel research over many sources, (4) old grep output that's no longer needed. Techniques: (a) clearing tool results, (b) subagents, (c) compaction with case facts, (d) structured notes.

A 1-c, 2-d, 3-b, 4-a.

Q Q5 · Calculation · Easy. An agent does 30 lookups. Trimming cuts each from 900 tokens to 120. How many tokens are saved in total? Why might the benefit be larger than the cost saving alone?

A (900 − 120) × 30 = **23,400 tokens**. And since each request resends earlier results, the savings repeat on every later turn, and less noise also helps the model focus.

Revision - Lesson 6

→ Context is finite: an attention budget, with recall declining as it fills.

→ Goal: the smallest possible set of high-signal tokens.

→ Progressive summarization loses amounts, dates and stated expectations.

→ Case facts: exact values in a structured block, outside summaries.

→ Trim tool results before they accumulate, in the tool or a PostToolUse hook.

→ Put key findings first, use section headers, and keep the query last.

→ Upstream agents return structured facts and metadata, not reasoning transcripts.

→ Just-in-time retrieval keeps references, and loads data when needed.

→ Compaction for chats, notes for milestones, subagents for parallel research, clearing for old tool output.

→ A larger window adds capacity, not attention.

→ Beta compaction requires appending response.content, including its blocks.

Your notes

Module Revision

Working with Claude - on one page

The API · One stateless endpoint. You own history. Append whole responses. Check stop_reason before content. A 200 can still be truncated or refused.

Prompts · A prompt is a specification. Use categorical criteria, not adjectives. Measure precision by category, and disable noisy ones while fixing them. Use 2-4 targeted examples, with reasoning. Long data first, task last.

Structured output · Schemas fix syntax, not truth. Nullable fields stop fabrication. Enums need other and unclear. Validate meaning in code. Retry with the document, the output and the error. Absent data needs a person.

tool_choice · auto may return text, any must call some tool, forced must call one. The exam tests this. Opus 5.5, Sonnet 5.5 and Fable 5.1 reject forced values today.

Thinking and effort · Adaptive thinking, with effort in output_config. Defaults differ by model. Hidden thinking is still billed. Treat upgrades as breaking changes.

Caching · An exact prefix match. One changing byte cancels it silently, so monitor cache_read_input_tokens.

Batches · 50% off, up to 24 hours, no latency SLA, no multi-turn tools, custom_id. Gap + 24 must fit the SLA.

Context · A budget. Exact facts stay outside summaries. Trim tools. Key findings go first. Move work out using notes and subagents.

Self-assessment

Answer without looking back. Each has a specific answer in this module.

#	Question
1	Why did the chatbot forget after scaling out?
2	What three things break if you store only the visible text?
3	Why is `content[0].text` a bug?
4	Which stop reasons must a production service handle?
5	How do you compute total input tokens?
6	Which HTTP statuses are retryable?
7	Why does "be conservative" fail? What works instead?
8	Give the three-step playbook for a noisy reviewer.
9	How many few-shot examples does the exam guide suggest? What must they show?
10	Which examples are worth writing, and how do you find them?
11	What's the "right altitude"?
12	Why don't XML tags stop prompt injection?
13	Why is schema-valid data sometimes wrong?
14	Compare `auto`, `any` and forced `tool_choice`. What's changed on current models?
15	Why do required fields cause fabrication?
16	What's the "other" + detail pattern for?
17	What three things go in a retry with feedback?
18	When won't a retry help?
19	What does `detected_pattern` enable?
20	Which settings return a 400 on Opus 5.5?
21	Does hidden thinking cost money?
22	What's Opus 5.5's default effort, and why does it matter?
23	What should you test before building a model cascade?

#	Question
24	State the caching write and read multipliers on Opus 5.5.
25	Name five silent cache invalidators.
26	How do you confirm a cache hit?
27	State five facts about the Batches API.
28	Why can't an agent loop run inside one batched request?
29	What submission gap fits a 30-hour SLA in the worst case?
30	How do you handle 400 oversized documents in a batch?
31	What's the difference between prompt and context engineering?
32	What goes in case facts?
33	Where can you trim tool results?
34	Choose a technique for: a long chat, multi-session coding, parallel research.
35	Why doesn't a 1M window remove the need for this lesson?

Where this module is used

Idea	Where it returns
`stop_reason` loop	AI06 L2, AI08 L3
Explicit criteria and false positives	AI12 L5, AI13 L5
Few-shot for tool choice and escalation	AI06 L3, AI06 L8
Structured output and nullable fields	AI05 L4, AI08 L5, AI13 L5
Forced `tool_choice`, exam versus today	AI06 L3, AI13 L3
Batches	AI05 L3, AI08 L6, AI13 L5
Prompt caching	AI05 L3, AI10 L5
Case facts and trimming	AI06 L9, AI13 L6
Notes and subagents	AI06 L6-L7, AI12

Working space